



DESIGN A RELIABLE PDF SEARCH AGENT

Software-Entwicklungspraktikum (SEP)

Sommersemester 2026

Offer

Customer

Technische Universität Braunschweig

Institute of Artificial Intelligence

Prof. Dr. Michel Besserve

Mühlenpfordtstraße 23

38106 Braunschweig

Supervisor: Baptiste Brémond

Contractor:

Name	E-Mail Address
Wajd Abdallah	w.abdallah@tu-raunschweig.de
Raghdaa Khoder	r.khoder@tu-braunschweig.de
Yosr Kharrat	y.kharrat@tu-braunschweig.de
Mouhib Hchaichi	m.hchaichi@tu-braunschweig.de

Braunschweig, 12. Juni 2026

Editor Overview

Chapter	Authors	Comments
1	Raghdaa Khoder	Checked by Yosr, Mouhib, Wajd
2	Raghdaa Khoder	Checked by Yosr, Mouhib, Wajd
3	Yosr Kharrat	Checked by Raghdaa, Mouhib, Wajd
4	Mouhib Hchaichi	Checked by Raghdaa, Yosr, Wajd
5	Wajd Abdallah	Checked by Raghdaa, Yosr, Mouhib
6	Yosr Kharrat, Mouhib Hchaichi	Checked by Raghdaa, Wajd
7	Raghdaa Khoder, Wajd Abdallah, Yosr Kharrat, Mouhib Hchaichi	Checked by all team members

Table of Contents

1 AI Statement	6
AI Statement	6
2 Introduction	7
2.1 Goal	7
2.2 Motivation	8
3 Formal Foundations	10
4 Project process	12
4.1 Milestones	12
4.2 Planned Process	13
5 Project Scope	15
5.1 Scope of Delivery	15
5.2 Cost Plan	15
5.3 Functional Scope	16
6 Development Guidelines	17
6.1 Configuration Management	17
6.2 Design and Programming Guidelines	18
6.3 Used Software	19
7 Project Organization	22
7.1 Interface with the client	22
7.2 Interface with Other Projects	22
7.3 Internal Communication	23

7.4 Team Roles and Responsibilities	23
8 Glossary	24

List of Figures

4.1	Gantt-Diagramm	14
-----	--------------------------	----

1 AI Statement

The AI tools used in this project are classified as **Type 0 –Assistive Support (Non-Generative)** according to the ISF AI Guidelines. This means that AI was only used for basic language support, such as spelling correction, grammar checking, and minor phrasing improvements, without adding new content, ideas, or technical decisions.

According to the ISF AI Guidelines, Type 0 usage does not require documentation or marking.

Braunschweig, 12. Juni 2026

Wajd Abdallah, Raghdah Khoder, Yosr Kharrat, Mouhib Hchaichi

2 Introduction

In modern information systems, large amounts of knowledge are stored in document-based formats, with PDF being one of the most widely used standards due to its portability and consistent layout. However, extracting and retrieving relevant information from PDF documents remains a challenging task, especially when dealing with large and heterogeneous document collections. Traditional keyword-based search approaches are often insufficient, as they fail to capture the semantic meaning of user queries and cannot effectively handle unstructured or complex document content.

At the same time, large language models (LLMs) have emerged as powerful tools for natural language understanding and question answering. Despite their capabilities, these models are prone to generating inaccurate or unverifiable information, commonly referred to as hallucinations. This limitation makes them unreliable when used in isolation, particularly in applications that require high accuracy and traceability.

The project Design a Reliable PDF Search Agent addresses this challenge by combining information retrieval with answer generation in a unified system. The core idea is to first identify relevant content from PDF documents and then generate responses based exclusively on this retrieved evidence. This approach follows the principle of retrieval-augmented generation, ensuring that generated answers are grounded in actual document content.

This document outlines the objectives, scope, and initial structure of the project, providing a foundation for the systematic development of a reliable and efficient PDF search system.

2.1 Goal

The goal of this project is to design and implement a reliable PDF search and question-answering system that enables users to efficiently retrieve and utilize information from document collecti-

ons. The system will process PDF files, extract textual content, and build an internal representation that supports efficient and accurate search operations.

A key objective is to enhance the reliability of generated answers by ensuring that all responses are grounded in retrieved evidence. Instead of relying solely on generative capabilities, the system will integrate information retrieval techniques with language models to provide verifiable and context-aware answers.

Furthermore, the system should be designed with robustness and scalability in mind, allowing it to handle diverse document structures and varying data quality. Usability is also an important aspect, as the system should provide clear and transparent results that allow users to trace answers back to their original sources.

Overall, the project aims to deliver a practical and reliable solution that meets the customer's requirements in terms of accuracy, performance, and user experience.

2.2 Motivation

The motivation for this project stems from the increasing reliance on large language models in information systems, particularly for tasks such as question answering and document analysis. While these models offer significant advantages in terms of flexibility and language understanding, their tendency to produce incorrect or unsupported information poses a serious limitation in real-world applications.

This issue becomes especially relevant when working with document-based data sources such as PDF files. These documents often contain complex layouts, unstructured text, and domain-specific information, making reliable information extraction and retrieval a non-trivial task. Users are frequently required to manually search through multiple documents, which is inefficient and prone to errors.

By combining document retrieval with controlled answer generation, this project aims to bridge the gap between powerful language models and trustworthy information systems. The resulting PDF search agent is intended to provide accurate, source-based answers while maintaining the flexibility of natural language interaction.

In addition, the project is conducted within the scope of the Software Engineering Project (SEP), offering the opportunity to apply theoretical concepts in a practical environment. It enables the development of skills in system design, implementation, and collaborative software engineering, while addressing a relevant and challenging real-world problem.

3 Formal Foundations

This chapter outlines the formal and technical foundations underlying the development of the PDF search agent. The system is based on concepts from information retrieval, natural language processing, and modern language model architectures.

A central component of the system is information retrieval (IR), which focuses on identifying relevant documents or text passages in response to a user query. Traditional IR approaches rely on keyword-based methods such as term frequency or inverted indices. While efficient, these approaches often fail to capture the semantic meaning of queries and document content.

To address this limitation, the system incorporates semantic search techniques, which represent text using vector embeddings. These embeddings allow the system to measure similarity between queries and document content on a semantic level, improving the relevance of retrieved results.

Another key foundation is the use of large language models (LLMs), which enable natural language understanding and answer generation. LLMs are capable of producing human-like responses, but they may generate incorrect or unverifiable information when used without external grounding.

To ensure reliability, the project follows the concept of retrieval-augmented generation (RAG). In this approach, the system first retrieves relevant information from the document collection and then uses this information as context for generating answers. This significantly reduces the risk of hallucinations and ensures that responses are based on actual document content.

In addition, the system relies on PDF text extraction techniques to convert document content into machine-processable text. Since PDF files may contain complex layouts and unstructured data, robust extraction methods are required to ensure data quality for subsequent processing steps.

From an implementation perspective, the system will be developed using Python, which provides extensive libraries for document processing, machine learning, and natural language processing. Version control and collaboration will be managed using GitLab to support a structured and traceable development process.

4 Project process

Here the milestones of the project are noted and the planned project timeline is presented in the form of a Gantt Chart.

4.1 Milestones

The milestones include official submissions and internal development goals to ensure continuous progress and feedback.

No.	Milestone	Deliverables	Due Date
0	Kick-Off Meeting	Project start, team organization	10.04
1	Offer Submission	Project scope, objectives, planning	20.04
2	Requirements Specification	Requirements document + acceptance test specification	11.05
3	Interim Presentation	Prototype demo + presentation slides	22.05
4	Specialist Design Document	Specialist Design Document	01.06
5	Technical Design Document	Technical Design Document	22.06
6	Test Documentation	Test protocols + Test specifications	06.07
7	TDSE (Final Presentation)	Final system + Complete documentation	16.07

4.2 Planned Process

The project began with the Kick-Off Meeting on 10.04, where the team aligned on roles, tools, and objectives.

The Offer Submission (20.04) defines the project scope and clarifies the goal of developing a reliable PDF-based RAG system. The Requirements Specification (11.05) establishes the system requirements and acceptance criteria, while the Interim Presentation (22.05) demonstrates the first functional prototype.

Afterwards, the Specialist Design Document (01.06) describes the functional design of the system, and the Technical Design Document (22.06) details the architecture, technologies, and implementation decisions.

The Test Documentation (06.07) evaluates the system using retrieval metrics, answer correctness, robustness tests, and efficiency analysis.

The project concludes with the TDSE Final Presentation (16.07), where the completed system and full documentation are delivered.

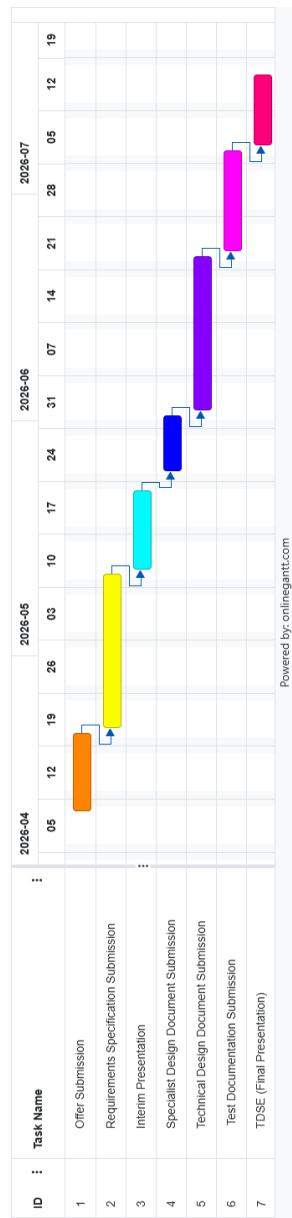


Figure 4.1: Gantt-Diagramm

5 Project Scope

This chapter defines the scope, deliverables, and objectives of the project. It outlines what will be delivered to the customer, the estimated effort and associated costs, as well as the core functionalities the application is expected to provide by the end of the development phase.

5.1 Scope of Delivery

At the end of the project, the following deliverables will be provided:

- A functional PDF search agent that allows users to query PDF documents and receive answers.
- The complete source code of the project, structured and documented, provided via a GitLab repository.
- A project documentation describing the architecture, methods (e.g., RAG), and implementation details.
- An evaluation of the system using appropriate metrics to assess retrieval and answer quality.
- A short user guide explaining how to install and use the application.

5.2 Cost Plan

The project is carried out by a team of four students, consisting of two Computer Science students and two Business Informatics students.

Each student is expected to earn 7 ECTS credits for the project. Since one ECTS credit corresponds to approximately 30 hours of workload, the total workload per student amounts to 210 hours.

For four students, this results in a total project effort of 840 working hours.

Assuming an hourly rate of 100 euros per hour, the total estimated cost of the project is therefore 84,000 euros.

5.3 Functional Scope

The system provides several core functionalities for interacting with PDF documents through a search-based approach.

- The user can upload one or multiple PDF documents to the system.
- The system processes and indexes the uploaded documents to enable efficient search.
- The user can enter natural language queries related to the content of the PDFs.
- The system retrieves relevant document sections based on the user's query.
- The system generates context-aware answers using the retrieved information.
- The system provides references or citations to the original document sections used for the answer.

6 Development Guidelines

This chapter outlines the development guidelines established for the project to ensure efficient collaboration, clean and consistent code, and a well-structured use of tools throughout the development process.

6.1 Configuration Management

This section describes the project's configuration management, including repository organization, development workflow, and versioning.

Source Code Repository

The project source code is managed in a Git repository hosted on the GITZ server¹. The branches and commits created by users are subject to the guidelines described below.

Branching and Integration

The project is developed according to the “Trunk-Based Development” model². The trunk branch is called `main` and always contains a correct and executable version of the project. Short-lived branches are created for features and bug fixes and are merged via merge requests only after review and basic validation.

¹https://git.rz.tu-bs.de/isf/sep/sep-2026/iai_drpsa_pdf-search-agent_g1/

²<https://trunkbaseddevelopment.com/>

Commit Guidelines

To keep the repository history understandable, commit messages should be short, precise, and meaningful. Commits should be atomic so that individual changes can be reviewed, tracked, or reverted more easily. Only working and consistent changes should be committed to the shared repository. Each commit contains a unique commit message and, if necessary, an additional description.

Versioning

The software is versioned according to the SemVer model³. Each version number consists of three parts: major, minor, and patch. A patch release introduces compatible bug fixes, a minor release introduces new functionality, and a major release may introduce breaking changes.

Changelog In addition to the code, a changelog is stored in the repository. It is saved in the CHANGELOG file and follows the “Keep a Changelog” format⁴.

6.2 Design and Programming Guidelines

This section defines the guidelines that apply to the source code in terms of language, coding style, and documentation format.

Language

Development is carried out in English. All code, comments, commit messages, and technical documentation are written in English to ensure consistency across the project.

³<https://semver.org/>

⁴<https://keepachangelog.com/en/1.1.0/>

Modular Design

The implementation follows a modular design. Core components such as PDF ingestion, parsing, chunking, indexing, retrieval, reranking, answer generation, and evaluation are separated as clearly as possible. This allows individual modules to be tested, compared, and replaced independently, which supports the reproducibility and reliability goals of the project.

Coding Style

Code should be written in a readable and maintainable style. Functions and classes should use descriptive names, avoid unnecessary complexity, and contain comments where additional explanation is needed. Reusable components should be preferred over duplicated code. Changes to the system should be implemented in a way that supports systematic testing and evaluation. Unnecessary complexity should be avoided unless it is justified by measurable improvements.

Documentation Format

All methods, classes, and other code elements are documented clearly and consistently. For Python code, documentation follows the conventions of PEP 257⁵.

6.3 Used Software

This section summarizes the software and tools used for the development of the project.

Programming Languages

The main implementation language of the project is Python⁶. Python is well suited for machine learning, natural language processing, and rapid prototyping. For documentation, LaTeX is used. This allows the team to create structured and consistent technical documents collaboratively.

⁵<https://peps.python.org/pep-0257/>

⁶<https://www.python.org/>

Project Dependencies

Depending on the final implementation, the project may use selected third-party libraries and frameworks for the following tasks:

- PDF parsing and text extraction
- embedding generation
- vector indexing and retrieval
- answer generation with large language models
- automated testing and evaluation
- metric computation and benchmark evaluation
- development of a lightweight user interface

Based on the initial project guidance, the implementation will favor stable and well-established open-source tools rather than commercial solutions or rapidly changing repositories. In particular, a combination such as `pypdf`⁷, `LangChain`⁸, and `RAGAS`⁹ provides a practical baseline with proven capabilities and known limitations.

This choice is based on the following reasons:

pypdf is a free and open-source Python library for working with PDF files and is suitable for document ingestion, since it can read PDFs and extract text and metadata.

LangChain is an open-source framework for building LLM-based applications and helps connect different parts of the pipeline, such as retrieval and generation, in a flexible and structured way.

RAGAS is designed to support the systematic evaluation of LLM applications and is therefore well suited for the reproducible evaluation pipeline required in this project.

⁷<https://pypdf.readthedocs.io/>

⁸<https://www.langchain.com/>

⁹<https://docs.ragas.io/>

Development Tools

The following tools are intended for development and project management:

- Visual Studio Code¹⁰ as the main development environment
- Git¹¹ for version control
- GitLab¹² for repository hosting and collaboration
- pytest¹³ for automated tests
- Jupyter Notebook¹⁴ for experiments and exploratory testing

Team Tools

To support team coordination and documentation, the following tools are used:

- Overleaf¹⁵ for collaborative LaTeX editing
- WhatsApp¹⁶ for internal coordination
- Lucidchart¹⁷ for diagrams and system architecture sketches
- OnlineGantt¹⁸ for creating and maintaining the Gantt chart for project timeline planning

¹⁰<https://code.visualstudio.com/>

¹¹<https://git-scm.com/>

¹²<https://about.gitlab.com/>

¹³<https://docs.pytest.org/>

¹⁴<https://jupyter.org/>

¹⁵<https://www.overleaf.com/>

¹⁶<https://www.whatsapp.com/>

¹⁷<https://www.lucidchart.com/>

¹⁸<https://www.onlinegantt.com/>

7 Project Organization

This chapter presents the communication structure and coordination mechanisms established for the project, ensuring efficient collaboration, clear responsibilities, and a smooth exchange of information throughout all project phases.

7.1 Interface with the client

Communication with the client (supervisor) is conducted through regular in-person meetings. These meetings are used to discuss project progress, address challenges, and receive guidance.

These meetings are typically held on Thursdays at 9:45 AM. During these meetings, we review our progress, validate our design decisions (for example, choices related to the RAG pipeline), and make sure the system meets requirements like reliability and reproducibility.

In addition to these weekly meetings, email is used for clarifications and follow-ups.

7.2 Interface with Other Projects

Another group is working on the same project topic. We are independent from each other, meaning we do not rely on them to complete our work.

However, staying in touch with them can be beneficial. By sharing ideas, discussing challenges, and exchanging experiences, both teams can learn and improve their solutions. This kind of contact is completely optional and is not necessary for the success of our project.

7.3 Internal Communication

The team communicates via platforms like WhatsApp, email, or similar tools. Weekly meetings are held to review progress and coordinate tasks. Communication is continuous and not limited to meetings:

Problems are discussed immediately via chat tools.

Technical decisions are documented.

Tasks are tracked using version control platforms (e.g., GitLab Issues).

We store all code and documents in GitLab, so everyone has the latest version.

This approach guarantees that every team member can reproduce evaluation runs, understand design trade-offs, and contribute effectively to building a measurable, reliable PDF search agent.

7.4 Team Roles and Responsibilities

To keep the project organized, each team member has clear tasks.

Project Coordinator – Yosr Kharrat

Yosr talks to the supervisor (the client). She reports how the project is going, and most importantly, she tells the supervisor when there are problems or difficulties. She also leads the weekly meetings on Thursdays.

Project Manager – Wajd Abdallah

Wajd plans the work and makes sure everything is done on time. He manages the GitLab repository, assigns tasks to team members, and checks that the team follows the project schedule and rules.

Testing and Diagrams – Raghda Khoder and Mouhib Hchaichi

All team members help with writing documentation and preparing the final presentation.

8 Glossary

Table 8.1: Glossary

No.	Technical Term	Definition
1	RAG	Retrieval-Augmented Generation. A method where relevant information is retrieved from documents before an answer is generated.
2	PDF Ingestion	The process of loading and preparing PDF documents in the system.
3	Parsing	The process of extracting text and structure from a file so that it can be processed further.
4	Chunking	The process of splitting a document into smaller text segments.
5	Retrieval	The process of finding relevant information or text passages for a query.
6	Reranking	The process of reordering retrieved results so that the most relevant ones appear first.
7	Citation	A reference showing which text passage or document supports an answer.
8	Abstention	The decision not to answer when there is not enough evidence in the documents.
9	Evaluation Pipeline	A structured process used to measure the quality and reliability of the system.
10	Reliability	The ability of the system to work correctly and consistently.